

INF560 / Unidad 3 / GL07

[GUÍA DE LABORATORIO](#)

CRUD con resource controller

Crear, leer, actualizar y eliminar eventos con las 7 rutas REST

Laravel 13 · REST · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 4 h

Métodos HTTP de esta guía: **GET** **POST** **PUT** **DELETE**

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

En la GL06, [eventos-app](#) ya lee los eventos desde PostgreSQL con paginación. Pero solo podemos **verlos**: no hay forma de crear, editar ni eliminar eventos desde la aplicación. En esta guía construimos el **CRUD completo** (Create, Read, Update, Delete) usando un **resource controller** y las siete rutas REST que Laravel genera automáticamente.

Objetivos de la guía

- Comprender las 7 acciones REST y generarlas con `Route::resource()`.
- Usar **route model binding** para inyectar el `Evento` por su id.
- Construir los formularios de creación y edición con `@csrf` y `@method`.
- Implementar `store`, `update` y `destroy`, con mensajes flash de confirmación.

Requisitos previos

- La GL06 completa: modelo `Evento`, base poblada con el seeder y `index` paginado funcionando.
- El layout de la GL04 (`<x-layout>`) para envolver las vistas.

ALCANCE DE ESTA GUÍA

El CRUD quedará funcional con una **validación mínima en línea** (para que guardar no falle). En la **GL08** moveremos esa validación a `Form Requests` con mensajes y errores en pantalla, y en una guía posterior agregaremos la subida de imagen. Aquí el foco es el **flujo CRUD y las rutas REST**.

01 Teoría breve: REST y resource controllers

1.1 Las 7 acciones REST

Un recurso (como `eventos`) se gestiona con siete acciones estándar. Cada una combina un **verbo HTTP** con una **URI** y se atiende con un método del controlador:

Verbo	URI	Acción	Nombre de ruta	Propósito
GET	/eventos	index	eventos.index	Listar todos
GET	/eventos/create	create	eventos.create	Form. de creación
POST	/eventos	store	eventos.store	Guardar el nuevo
GET	/eventos/{evento}	show	eventos.show	Ver uno
GET	/eventos/{evento}/edit	edit	eventos.edit	Form. de edición
PUT	/eventos/{evento}	update	eventos.update	Actualizar uno
DELETE	/eventos/{evento}	destroy	eventos.destroy	Eliminar uno

1.2 Route::resource lo genera todo

En vez de escribir siete rutas a mano, una sola línea las declara todas y las conecta con los métodos del controlador que llevan esos mismos nombres.

1.3 Route model binding

DEFINICIÓN — ROUTE MODEL BINDING

Si un método del controlador declara un parámetro con el **tipo del modelo** (`show(Evento $evento)`), Laravel toma el `{evento}` de la URL, busca ese registro por su `id` y se lo entrega ya cargado. Si no existe, responde **404** automáticamente. Se acabó el `Evento::find($id)` manual.

02 Ejercicio 1 — La ruta de recurso y el controlador

Paso 1 — Declarar la ruta de recurso

En `routes/web.php`, reemplace la ruta suelta de `index` por la ruta de recurso:

```
routes/web.php Copiar

use App\Http\Controllers\EventoController;

Route::resource('eventos', EventoController::class);
```

Paso 2 — Ver las 7 rutas generadas

```
terminal Copiar

$ php artisan route:list --name=eventos

GET|HEAD eventos ..... eventos.index
POST eventos ..... eventos.store
GET|HEAD eventos/create ..... eventos.create
GET|HEAD eventos/{evento} ..... eventos.show
PUT|PATCH eventos/{evento} ..... eventos.update
GET|HEAD eventos/{evento}/edit .... eventos.edit
DELETE eventos/{evento} ..... eventos.destroy
```

Paso 3 — Completar el controlador

El `EventoController` ya tiene `index()` de la GL06. Agregue los métodos restantes; los iremos llenando en los siguientes ejercicios. Este es el esqueleto:

```
app/Http/Controllers/EventoController.php Copiar

<?php

namespace App\Http\Controllers;

use App\Models\Evento;
use Illuminate\Http\Request;
use Illuminate\Support\Str;

class EventoController extends Controller
{
    public function index() { /* GL06 */ }

    public function create() {}
    public function store(Request $request) {}
    public function show(Evento $evento) {}
    public function edit(Evento $evento) {}
    public function update(Request $request, Evento $evento) {}
    public function destroy(Evento $evento) {}
}
```

NOTA

Note el tipo `Evento $evento` en `show`, `edit`, `update` y `destroy`: ahí actúa el route model binding.

03 Ejercicio 2 — Mostrar un evento (show)

Empezamos por la lectura de un evento individual, que es la acción más sencilla y estrena el route model binding.

Paso 1 — El método show

```
EventoController.php

public function show(Evento $evento)
{
    return view('eventos.show', compact('evento'));
}
```

Paso 2 — La vista de detalle

```
resources/views/eventos/show.blade.php

<x-layout>
    <article class="max-w-2xl mx-auto space-y-3">
        <h1 class="text-2xl font-bold">{{ $evento->titulo }}</h1>
        <p class="text-gray-500">
            {{ $evento->fecha->format('d/m/Y H:i') }} · {{ $evento->lugar }}
        </p>
        <p>{{ $evento->descripcion }}</p>
        <p class="font-medium">
            Cupo: {{ $evento->cupo }} · Precio: Bs {{ $evento->precio }}
        </p>

        <a href="{{ route('eventos.index') }}" class="text-blue-600">Volver</a>
    </article>
</x-layout>
```

Paso 3 — Enlazar desde la lista

En `eventos/index.blade.php`, haga que el título de cada tarjeta lleve al detalle:

```
resources/views/eventos/index.blade.php

<a href="{{ route('eventos.show', $evento) }}" class="font-semibold hover:underline">
    {{ $evento->titulo }}
</a>
```

NOTA

Fíjese que a `route('eventos.show', $evento)` le pasamos el **objeto** completo: Laravel extrae su `id` solo. Visite `/eventos/1` para probarlo, y `/eventos/999` para ver el 404 automático.

04 Ejercicio 3 — Crear un evento (create y store)

Paso 1 — Mostrar el formulario (create)

```
EventoController.php

public function create()
{
    return view('eventos.create');
}
```

```
}
```

Paso 2 — El formulario

El formulario envía por **POST** a `eventos.store`. La directiva `@csrf` agrega un token que protege contra falsificación de peticiones (sin él, Laravel responde [419](#)).

resources/views/eventos/create.blade.php

Copiar

```
<x-layout>
  <form action="{{ route('eventos.store') }}" method="POST" class="max-w-xl space-y-4">
    @csrf

    <div>
      <label class="block text-sm font-medium">Título</label>
      <input type="text" name="titulo" value="{{ old('titulo') }}"
        class="mt-1 w-full rounded-lg border-gray-300">
    </div>

    <div>
      <label class="block text-sm font-medium">Descripción</label>
      <textarea name="descripcion" rows="4"
        class="mt-1 w-full rounded-lg border-gray-300">{{ old('descripcion')
    }}</textarea>
    </div>

    <div>
      <label class="block text-sm font-medium">Fecha y hora</label>
      <input type="datetime-local" name="fecha" value="{{ old('fecha') }}"
        class="mt-1 w-full rounded-lg border-gray-300">
    </div>

    <div>
      <label class="block text-sm font-medium">Lugar</label>
      <input type="text" name="lugar" value="{{ old('lugar') }}"
        class="mt-1 w-full rounded-lg border-gray-300">
    </div>

    <div class="grid grid-cols-2 gap-4">
      <div>
        <label class="block text-sm font-medium">Cupo</label>
        <input type="number" name="cupo" value="{{ old('cupo') }}"
          class="mt-1 w-full rounded-lg border-gray-300">
      </div>
      <div>
        <label class="block text-sm font-medium">Precio (Bs)</label>
        <input type="number" step="0.01" name="precio" value="{{ old('precio', 0) }}"
          class="mt-1 w-full rounded-lg border-gray-300">
      </div>
    </div>

    <label class="flex items-center gap-2">
      <input type="checkbox" name="publicado" value="1">
      <span class="text-sm">Publicado</span>
    </label>

    <button type="submit" class="rounded-lg bg-blue-600 px-4 py-2 text-white">
      Guardar evento
    </button>
  </form>
</x-layout>
```

```
</form>
</x-layout>
```

Cada `input` conserva su valor con `old('campo')`, para que si la validación falla el usuario no pierda lo escrito.

Paso 3 — Guardar (store)

```
EventoController.php Copiar

public function store(Request $request)
{
    // Validación mínima (se ampliará en la GL08)
    $datos = $request->validate([
        'titulo'      => 'required|string|max:255',
        'descripcion' => 'required|string',
        'fecha'       => 'required|date',
        'lugar'       => 'required|string|max:255',
        'cupos'       => 'required|integer|min:1',
        'precio'      => 'required|numeric|min:0',
    ]);

    $datos['slug']      = Str::slug($datos['titulo']);
    $datos['publicado'] = $request->boolean('publicado');

    Evento::create($datos);

    return redirect()->route('eventos.index')
        ->with('exito', 'Evento creado correctamente.');
```

Tres detalles del método:

- `$request->validate([...])` revisa los datos y, si algo falla, regresa al formulario con los errores.
- `Str::slug()` genera el slug a partir del título; `$request->boolean()` convierte el checkbox en `true/false`.
- `redirect()->route()->with()` vuelve a la lista y deja un mensaje flash de un solo uso.

Paso 4 — Mostrar el mensaje flash

En el layout (`components/layout.blade.php`), justo antes del `{{ $slot }}`, muestre el aviso:

```
resources/views/components/layout.blade.php Copiar

@if (session('exito'))
    <div class="mb-4 rounded-lg bg-green-100 px-4 py-2 text-green-800">
        {{ session('exito') }}
    </div>
@endif
```

05 Ejercicio 4 — Editar un evento (edit y update)

Paso 1 — Mostrar el formulario con datos (edit)

```
EventoController.php Copiar

public function edit(Evento $evento)
{
    return view('eventos.edit', compact('evento'));
}
```

Paso 2 — El formulario de edición

Es casi igual al de creación, con tres diferencias clave: apunta a `eventos.update`, incluye `@method('PUT')` y llega con los valores precargados.

```
resources/views/eventos/edit.blade.php Copiar

<form action="{{ route('eventos.update', $evento) }}" method="POST" class="max-w-xl space-y-4">
    @csrf
    @method('PUT')

    <div>
        <label class="block text-sm font-medium">Título</label>
        <input type="text" name="titulo"
            value="{{ old('titulo', $evento->titulo) }}"
            class="mt-1 w-full rounded-lg border-gray-300">
    </div>

    <div>
        <label class="block text-sm font-medium">Fecha y hora</label>
        <input type="datetime-local" name="fecha"
            value="{{ old('fecha', $evento->fecha->format('Y-m-d\TH:i')) }}"
            class="mt-1 w-full rounded-lg border-gray-300">
    </div>

    {{-- ...resto de campos igual, con old('campo', $evento->campo)... --}}

    <label class="flex items-center gap-2">
        <input type="checkbox" name="publicado" value="1"
            @checked(old('publicado', $evento->publicado))>
        <span class="text-sm">Publicado</span>
    </label>

    <button type="submit" class="rounded-lg bg-blue-600 px-4 py-2 text-white">
        Actualizar evento
    </button>
</form>
```

¿POR QUÉ @METHOD('PUT')?

Los formularios HTML solo entienden `GET` y `POST`. Para usar `PUT` (o `DELETE`), el formulario se envía como `POST` y `@method('PUT')` agrega un campo oculto que le dice a Laravel el verbo real. Se llama **method spoofing**. Y `@checked(...)` —que ya vio en la GL04— marca el checkbox si el valor es verdadero.

Paso 3 — Actualizar (update)

```
EventoController.php Copiar

public function update(Request $request, Evento $evento)
{
    $datos = $request->validate([
        'titulo'      => 'required|string|max:255',
        'descripcion' => 'required|string',
        'fecha'       => 'required|date',
        'lugar'       => 'required|string|max:255',
        'cupos'       => 'required|integer|min:1',
        'precio'      => 'required|numeric|min:0',
    ]);
}
```

```

$datos['slug']      = Str::slug($datos['titulo']);
$datos['publicado'] = $request->boolean('publicado');

$evento->update($datos);

return redirect()->route('eventos.index')
    ->with('exito', 'Evento actualizado correctamente.');
```

EVITE DUPLICAR EL FORMULARIO

Los formularios de crear y editar comparten casi todos los campos. Como aprendió en la GL04, puede extraerlos a un parcial `eventos/_form.blade.php` e incluirlo con `@include` en ambas vistas, pasando la ruta y el verbo. Quedará como reto en el trabajo independiente.

06 Ejercicio 5 — Eliminar un evento (destroy)

Paso 1 — El método destroy

```

EventoController.php
Copiar

public function destroy(Evento $evento)
{
    $evento->delete();

    return redirect()->route('eventos.index')
        ->with('exito', 'Evento eliminado correctamente.');
```

Paso 2 — El botón de eliminar

No se puede eliminar con un enlace: **DELETE** exige un formulario. Agregue este mini-formulario a cada tarjeta en la lista, junto a un enlace para editar:

```

resources/views/eventos/index.blade.php
Copiar

<div class="mt-3 flex items-center gap-3">
    <a href="{{ route('eventos.edit', $evento) }}" class="text-sm text-blue-600">
        Editar
    </a>

    <form action="{{ route('eventos.destroy', $evento) }}" method="POST"
        onsubmit="return confirm('¿Eliminar este evento?')">
        @csrf
        @method('DELETE')
        <button type="submit" class="text-sm text-red-600">Eliminar</button>
    </form>
</div>
```

CONFIRME ANTES DE BORRAR

El `onsubmit="return confirm(...)"` pide confirmación al usuario antes de enviar. `delete()` borra el registro de la base de forma permanente; no hay papelera. En proyectos reales se usa a veces el **soft delete** para conservar el dato marcándolo como eliminado, tema para más adelante.

LO QUE ACABA DE LOGRAR

Ya tiene el ciclo completo: listar, ver, crear, editar y eliminar eventos desde la interfaz, con las 7 rutas REST y mensajes de confirmación. `eventos-app` es ahora una aplicación funcional de gestión. En la **GL08** endureceremos los formularios: validación con Form Requests, mensajes de error por campo y reglas más estrictas.

07 Trabajo independiente

Construya el CRUD completo del recurso `categorias` (modelo creado en la GL06), aplicando todo lo de esta guía.

Consigna

- Declare `Route::resource('categorias', CategoriaController::class)` y verifique con `route:list`.
- Implemente las 7 acciones en `CategoriaController` (campos: `nombre`, `slug` auto, `descripcion`).
- Cree las vistas `index`, `create`, `edit` y `show`, con mensajes flash.
- **Reto:** extraiga el formulario a un parcial `categorias/_form.blade.php` reutilizado por `create` y `edit`.

Entregables

- Commits atómicos por acción, y push a `INF560-2026-02-Apellido`.
- Capturas del CRUD funcionando (crear, editar, eliminar) y del parcial reutilizado.

08 Checklist de verificación

- ☐ `Route::resource` declarada y `route:list` muestra las 7 rutas de eventos.
- ☐ `show` usa `route model binding` y la vista de detalle funciona (404 en id inexistente).
- ☐ El formulario de creación tiene `@csrf` y `old()` en cada campo.
- ☐ `store` valida, genera el slug, guarda y redirige con mensaje flash.
- ☐ El mensaje flash se muestra en el layout.
- ☐ El formulario de edición usa `@method('PUT')` y llega precargado con `@checked`.
- ☐ `update` modifica el registro y redirige con confirmación.
- ☐ El botón eliminar usa un formulario con `@method('DELETE')` y confirma antes de borrar.
- ☐ Trabajo independiente: CRUD de categorías con formulario en parcial, en GitHub.

09 Problemas frecuentes

Síntoma	Causa y solución
419 Page Expired	Falta <code>@csrf</code> en el formulario. Agréguelo justo después de la etiqueta <code><form></code> .
405 Method Not Allowed	El formulario de edición/borrado no lleva <code>@method('PUT')</code> o <code>@method('DELETE')</code> .
404 Not Found	El id no existe (<code>route model binding</code> no lo encontró) o la ruta no está declarada.
Route [eventos.store] not defined	Falta <code>Route::resource</code> , o escribió mal el nombre de la ruta.
Add [slug] to fillable	El modelo no tiene 'slug' en <code>\$fillable</code> . Revíselo (GL06).

Síntoma	Causa y solución
Unique violation: slug	Dos eventos con el mismo título generan el mismo slug. La validación unique llega en la GL08.
El checkbox no guarda false	Use \$request->boolean('publicado'): un checkbox sin marcar no envía nada.

10 Comandos de referencia

comandos usados en esta guía	Copiar
<pre># Rutas php artisan route:list --name=eventos # ver las rutas del recurso # Generar un controlador de recurso (si lo crea de cero) php artisan make:controller CategoriaController --resource --model=Categoria # Directivas Blade usadas en los formularios @csrf # token de seguridad (obligatorio en POST) @method('PUT') # simula el verbo PUT @method('DELETE') # simula el verbo DELETE @checked(condicion) # marca un checkbox old('campo', \$valor) # recupera el valor tras un error</pre>	

11 Rúbrica de evaluación

Criterio	Pts
Route::resource y controlador con las 7 acciones	15
show con route model binding y vista de detalle	10
create + store: formulario con @csrf, slug y flash	20
edit + update: method spoofing y datos precargados	20
destroy: formulario DELETE con confirmación	15
Mensajes flash mostrados en el layout	5
Trabajo independiente: CRUD de categorías con parcial	15
Total	100